

China Construction Bank technical code coding and naming standards (2026 Edition)

September 2026

Table of Contents

1. Introduction 5
   1.1 Purpose of writing 5
   1.2 Scope of application 5
2. Technical code coding standards 6
   2.1 Business function coding 6
      2.1.1 Uses of business function coding 6
      2.1.2 Business function coding rules 6
   2.2 Application domain coding 6
      2.2.1 Application domain: Uses of coding 6
      2.2.2 Application domain coding rules 7
   2.3 Application system coding 7
      2.3.1 Applications of system coding 7
      2.3.2 Apply system coding rules 7
   2.4 Subsystem encoding 7
      2.4.1 The purpose of subsystem encoding 7
      2.4.2 Subsystem coding rules 8
   2.5 Service module coding 8
      2.5.1 The purpose of service module encoding 8
      2.5.2 Service module coding rules 8
   2.6 AI agent encoding 9
      2.6.1 Uses of agent encoding 9
      2.6.2 Agent encoding rules 9
   2.7 AI Model Context Protocol (MCP) encoding 9
      2.7.1 The purpose of Model Context Protocol (MCP) encoding . 9
      2.7.2 Model Context Protocol (MCP) encoding rules 9
   2.8 AI skills coding 9
      2.8.1 Uses of skills encoding 10
      2.8.2 Skills coding rules 10
   2.9 Response code encoding 10
      2.9.1 The purpose of response code encoding 10
      2.9.2 Four-segment twelve-bit encoding rule 10
      2.9.3 Response code code for successful transactions 11
   2.10 Error code encoding 13

2.10.1 Uses of error code encoding 13
2.10.2 Error code encoding rules 13
2.10.3 Rules for classifying first-level errors 14
2.10.4 Error Second-Level Classification Rules 15
2.10.5 Examples of secondary classification of errors 17
2.10.6 Principles for writing misinformation 25
2.10.7 Error information escape mechanism 26
2.11 Serial number (global event tracking number) code 26
2.11.1 The purpose of serial numbers (global event tracking numbers) codes 27
2.11.2 Serial numbers (global event tracking numbers) are generated independently 27
2.11.3 Serial number (global event tracking number) coding rules 27
2.11.4 Serial number requirements in serial numbers (global event tracking numbers) coding 30
2.12 Types and codes of transactions..... 31
2.12.1 The purpose of the type of transaction 31
2.12.2 Classification of transaction types 31
2.12.3 Transaction type coding rules 34
3. Technical code naming standards..... 36
3.1 Six basic rules for naming identifiers 36
3.1.1 [Naming Rule 1] Word Usage Rules 36
3.1.2 [Naming Rule 2] Length Rule 37
3.1.3 [Naming Rule 3] Spacing rules 37
3.1.4 [Naming Rule 4] Case Rules 37
3.1.5 [Naming Rule 5] Number Rules 37
3.1.6 [Naming Rule 6] Conformance Rule 38
3.2 Naming of programming identifiers 38
3.2.1 Module 38
3.2.2 Classes/interfaces 39
3.2.3 Function/Method 39
3.2.4 Variables/parameters 40
3.2.5 Constant 40
3.2.6 C/C++ proprietary identifier 40
3.2.7 Macro 40
3.2.8 JAVA proprietary identifier 41
3.3 Naming of message identifiers..... 42
3.3.1 Packet header identifier 42
3.3.2 Message text identifier 43
4. Coding standards for multilingual and multi-entity codes..... 43
4.1 Coding standards for multiple language codes 43
4.2 Coding standards for multi-entity codes 46

1.Introduction

1.1 Purpose of writing

This specification specifies and encodes the objects that need to be referenced within the entire framework framework. It does not enforce local variables; it is recommended to use the development specification for constraints.

1.2 Scope of application

Each project team developer should carry out related design and coding work in accordance with this specification.

2. Technical code coding standards

2.1 Business function coding

2.1.1 Uses of business function coding

Business functions are instantiated asset objects formed by analyzing and restoring four-level tasks (CPC restore) based on factors such as customer products, channels, during system development and implementation. Coding business functions is meant to uniquely identify these instantiated objects.

2.1.2 Business function coding rules

The business function encoding rule uses a three-segment**, 10-digit** encoding method: the first **segment has 6** digits corresponding to the three-level active code, **the second segment has 1** digit as the underline, and the third segment has **4** digits of the numeric serial number. When applying to add a business function in the system, the sequence number is automatically generated by the system (the same applies to the serial numbers mentioned in the following sections). The encoding is globally unique

2.2 Application domain coding

2.2.1 Application domain: Uses of coding

The new architecture metamodel defines new architectural object concepts such as application domains, application systems, subsystems, and service modules. Coding application domains is meant to uniquely identify them.

2.2.2 Application domain coding rules

Application domains use a 5-digit encoding system, consisting of uppercase "D" + 4 digits, globally unique. Once created and used, they will not be renumbered due to type changes, application system changes, or correspondence with other assets.

2.3 Application system coding

2.3.1 Applications of system coding

The new architecture meta-model defines new architectural object concepts such as application domains, application systems, subsystems, and service modules. Coding application systems is intended to uniquely identify them.

2.3.2 Apply system coding rules

Application system encoding uses **a** 5-digit code, consisting of the uppercase letter "**S**" and **4** digits. It is globally unique. Once created and used, it will not be renumbered due to changes in type, included subsystems, or changes in mapping relationships with other objects. If the system goes offline, its number is reclaimed, and without conflicts or confusion, the number can be reassigned and used

2.4 Subsystem encoding

2.4.1 The purpose of subsystem encoding

The new architecture metamodel defines new architectural object concepts such as application domains, application systems, subsystems, and service modules. Coding subsystems is meant to uniquely identify them.

2.4.2 Subsystem coding rules

During the implementation of architectural recheck results, the subsystem baseline is a transitional state. First, the subsystem fully inherits the existing physical subsystem and gradually integrates it to form an end-to-end target state subsystem. The subsystem uses **a** 6-bit encoding method, consisting of the uppercase letter "**W**" and 5 digits, globally unique. Once created and used, it will not be renumbered due to changes in type or mapping relationships with other objects. If the subsystem is taken offline, its numbering is reclaimed, and without conflicts or confusion, the number can be reassigned and used. The existing subsystem encoding remains unchanged, and the new 5-digit numbers for the subsystem are encoded in a sequential ascending manner

2.5 Service module coding

2.5.1 The purpose of service module encoding

The new architecture meta-model defines new architectural object concepts such as application domains, application systems, subsystems, and service modules. The encoding of service modules is intended to uniquely identify them.

2.5.2 Service module coding rules

The service module encoding uses **a** 6-digit format, consisting of the uppercase letter "**M**" and **5** digits plus uppercase letters. It is globally unique. Once created and used, it will not be renumbered due to changes in type or mapping relationships with other objects. If the service module is taken offline or discontinued, its number will be reclaimed, and without conflicts or confusion, the number can be reassigned and used

2.6 AI agent encoding

2.6.1 Uses of agent encoding

With the rapid development and widespread application of artificial intelligence (AI), it is necessary to encode agents, model context protocols (MCP), and skills, so that each relatively independent object can be uniquely identified and globally unique, facilitating refined management and convenient use

2.6.2 Agent encoding rules

The agent uses **a** three-segment**,** 8-bit encoding, with the encoding rule is: **"AGT"** + **5** digits of a number or uppercase letter, with a globally unique code

2.7 AI Model Context Protocol (MCP) encoding

2.7.1 The purpose of Model Context Protocol (MCP) encoding

With the rapid development and widespread application of artificial intelligence (AI), it is necessary to encode agents, model context protocols (MCP), and skills, so that each relatively independent object can be uniquely identified and globally unique, facilitating refined management and convenient use

2.7.2 Model Context Protocol (MCP) encoding rules

The model context protocol uses two **segments of** 8-bit encoding, with the encoding rule being: **"MCP"** + **5** digits of a number or uppercase letter, with the code globally unique

2.8 AI skills coding

2.8.1 Uses of skills encoding

With the rapid development and widespread application of artificial intelligence (AI), it is necessary to encode agents, model context protocols (MCP), and skills, so that each relatively independent object can be uniquely identified and globally unique, facilitating refined management and convenient use

2.8.2 Skills coding rules

The skill uses two segments **of** 8-digit encoding, with the encoding rule being: **"SKL"** + **5** digits of a number or uppercase letter, with a globally unique code

2.9 Response code encoding

A response code is a string of codes previously agreed upon by both parties to represent feedback information. It can be either a number or a capital letter.

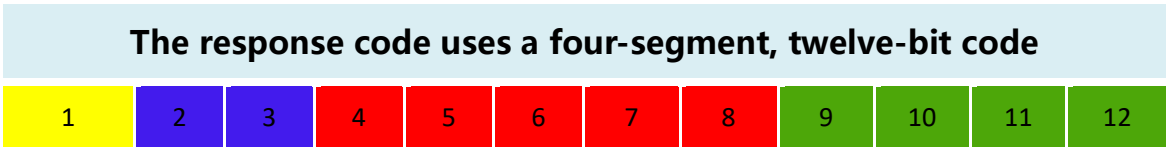
2.9.1 The purpose of response code encoding

The encoding of response codes is used to classify feedback from both parties, making it easy to identify quickly.

2.9.2 Four-segment twelve-bit encoding rule

The characters in the response code must use standard **UTF-8** encoding

The response code uses **a** 4-segment**, 12-bit** encoding method, as shown in the diagram below:



Primary classification	Secondary classification	Three-level classification	Serial number
1 person	2nd place	5	4
	Capital letters	Logic component encoding	Numbers or uppercase letters

First paragraph: 1 bit long, indicating the primary classification of the response code

Second paragraph: **2** bits long, indicating the secondary classification of the response code

Third paragraph: 5 characters long, indicating the three-level classification of response codes, which classifies response sources and requires filling in the corresponding logical component codes

Fourth paragraph: **4** bits long, indicating the sequence number of the response code

2.9.3 Response code for successful transactions

According to this specification, the response code for successful transactions generally uses **12** zeros

The 12 zero response codes can only be used to indicate successful transactions; this is the most obvious sign of a successful transaction and must not be used to convey other meanings

If a successful transaction requires a return prompt or warning message, it can be reflected via a response code.

For successful transaction response codes, the general encoding rules are retained, using **a** 4-segment**, 12-bit** encoding method, as shown in the

figure below:

The response code for successful transactions uses a four-segment, twelve-digit code											
1	2	3	4	5	6	7	8	9	10	11	12
Primary classification	**Secondary classification**	**Three-level classification**						**Serial number**			
1 person	**2nd place**	**5**						**4**			
I/O	**Capital letters**	**Logic component encoding**						**Numbers or uppercase letters**			

First paragraph: **length is 1** bit, indicating the primary classification of successful transaction response codes. Currently, there are two categories: prompt information (using capital **letters I<info>**); warning information class (using **capital letters W<warning>**).

Second paragraph: 2 characters long, representing the secondary classification of the successful response code. If there is no detailed secondary classification, you can directly fill in the two-digit ″**OK**" letters

Third paragraph: 5 bits long, indicating the three-level classification of the success response code, which classifies the successful response source and requires the corresponding logical component code

Paragraph 4: **4** bits long, indicating the serial number of the successful response code

2.10 Error code encoding

Error codes are a type of response code and are not used for incorrect physical positioning

2.10.1 Uses of error code encoding

The encoding of error codes is used to classify error information from both parties, making it easier to quickly identify errors.

2.10.2 Error code encoding rules

Characters in error codes must use standard **UTF-8** encoding

Error codes follow the encoding rules of response codes, using **a** 4-segment**, 12-bit** encoding method, as shown in the figure below:

Error codes use four-segment, twelve-digit codes											
1	2	3	4	5	6	7	8	9	10	11	12
Primary classification	**Secondary classification**		**Three-level classification**					**Number**			
Orientation	**Qualitative**		**Source of error**					**Error segmentation**			
1 person	**2nd place**		**5**					**4**			
XYZ	**Capital letters**		**Framework Unit Coding (Logic Subsystem Coding)**					**Numbers or uppercase letters**			

Paragraph 1: **length is 1** bit, used for error orientation (determining

the general direction of error handling) is a first-level classification of errors

Paragraph 2: 2 characters long, used to characterize errors (determine the nature or severity of errors) and indicate secondary classifications of errors

Third paragraph: **5** bits long, used for classifying and identifying error sources. Fill in the corresponding code for the logical component with the error

Fourth paragraph: 4 bits long, used for subdividing errors. Defined by each application (component).

Generally, since uppercase "**O**" and the number "**0**" can cause visual confusion, it is recommended to avoid or minimize the use of uppercase "**O**" as an encoding element to avoid **simultaneous occurrence of both "O" and "0" within the same encoding** system

2.10.3 Rules for classifying first-level errors

Currently, there are five types of errors at the primary level: technical error X, business error Y, third-party error Z, and successful transaction response codes I and W.

Technical errors:

Abnormal errors caused by technical reasons (such as platforms, operating systems, middleware, etc.).

Technical errors can be divided into two categories: self-healing errors and non-healing errors.

Self-healing error refers to a successful transaction that requires no technical processing, and a transaction will succeed after a short period. It is usually caused by a single transaction rejection caused by system mechanisms such as traffic control common components.

Non-healing error: A system error that fails even if the relevant

technical processing is not performed before trading.

For such errors, write "X" in the first digit of the error code

Business error:

Errors caused by non-compliance with business processing rules during business processing.

For such errors, write "Y" in the first digit of the error code

Third-party errors:

Errors returned by calling third-party systems (external third-party interface services).

For such errors, the first digit of the error code should be "**Z**"

Successful transaction response code:

If a successful transaction requires a return prompt or warning message, it will be reflected through the successful transaction response code.

The first digit of the prompt error code should be "I", and the first digit of the warning error code should be "W"

2.10.4 Error Second-Level Classification Rules

The secondary classification of error codes is a rapid qualitative of errors, helping to quickly determine the nature of errors and efficiently handle system faults. The primary classification is divided by nature, as shown in the table below:

Mistaken first-level classification	Primary classification code	Matching order	Secondary classification of errors	Secondary classification code
Business error	Y	1	Passwords and	PW (Password)

			authentication	
	Y	2	Quantity and limits	AL (Amount Limitation)
	Y	3	Permission control	AC (Authority Control)
	Y	4	Data content is relevant	DC (Data Context)
	Y	5	Content error	CE (Content Error)
	Y	6	Repeat transactions	DO Duplicate Operation)
	Y	7	Time and deadlines	TM (Time)
	Y	8	Risk control	RC (Risk Control)
	Y	9	Business logic	BL (Business Logic)
Technical errors	X	1	Uncertain trading results	NA(N/A)
	X	2	Hardware errors	HW (Hardware)
	X	3	Data content is relevant	DC (Data Context)
	X	4	Reading and writing	IO(I/O)
	X	5	Database-related	DB (Database)
	X	6	Network communication	NC (Network Communication)
	X	7	Security services	SS (Security Service)
	X	8	Component modules	CM (Components or Modules)
	X	9	Flow control	FC (Flow Control)

	X	10	Repeat transactions	DO (Duplicate Operation)
	X	11	Technical logic	TL (Technical Logic)
	X	12	Purely technical error	AT (Absolute Technique)
Third party Wrong	Z	1	Business logic	BL (Business Logic)
	Z	2	Technical logic	TL (Technical Logic)
Successful transaction response code	I	1	Note: If there is no detailed secondary classification, you can simply fill in the two letters "OK".	
	W	1		

For business class errors, if it is impossible to clearly distinguish which subclass they belong to, they are all classified as "Business Logic BL (Business Logic)" class errors; For technical class errors, if it is impossible to clearly distinguish which subclass they belong to, they are all classified as "Technical Logic TL (Technical Logic)" class errors

2.10.5 Examples of secondary classification of errors

Below is an example of the secondary classification of "business errors" and "technical errors."

The table below lists detailed examples of representative secondary classification of errors. Each project team should refer to the example list below to select the corresponding secondary classification code for "error messages."

Examples of secondary classifications for business-type errors

Secondary Classification of Business Errors (1)	Secondary classification code	**PW (Password)**
	Passwords and authentication	Password error limit / authentication error limit is exceeded
		Password error (password verification failed) / verification code failed
		The password length is insufficient
		The password has been locked or the initial password status is abnormal
		Password invalid/password locked
		Password/password has expired
		Errors that may occur in various authentication methods (certificates, fingerprints).
		Others

Secondary Classification of Business Errors (2)	Secondary classification code	**AL (Amount Limitation)**
	Quantity and limits	Quantity or amount not within the limit (not meeting or exceeding the limit)
		If the minimum deposit amount is not reached
		The maximum limit is full; no additional or increased limits are allowed
		Insufficient balance
		The amount does not meet the standard (the amount must be an integer, without dime points; or it must be a multiple of 50 yuan)
		If the refund amount exceeds the original payment amount, it needs to be checked
		The payment amount must be no less than the outstanding amount
		The number of transactions and operations exceeds the limit
		Others

classification of business-type	Secondary classification code	**AC (Authority Control)**
	Permission	Insufficient permissions (such as not meeting user level)
		Lack of permission review information or necessary authentication media prohibits transactions

control	No right to operate others' accounts
	Skipping levels
	Others

Secondary Classification of Business Errors (4)

Secondary classification code	**DC (Data Context)**
Data content is relevant	No relevant records (e.g., no detailed records, no relevant billing information)
	Unable to find the corresponding account information
	Channel information does not exist (e.g., the customer has not signed a contract)
	Matching data items cannot be found, and corresponding content cannot be found
	No relevant information or data, or no business parameter settings
	The amount of data meeting the requirements is enormous, requiring confirmation or change of data return methods
	If there is more than one or a specified number of pieces of information or data, it is not possible to determine which one or how many to use
	Others

Secondary classification of business-type errors (5)

Secondary classification code	**CE (Content Error)**
Content error	Incorrect formatting, exceeding scope, or missing necessary information (such as account type, payment type, currency, etc. not meeting standards)
	Related data verification failures (e.g., non-compliant account bits, incorrect account format, incorrect track format)
	Bank card last-digit verification failed, 18-digit ID card last-digit verification failed
	Incorrect ID number format, phone number not 11 digits, incorrect date format (e.g., 20120230)
	Mismatch in status or information (for example, request information does not match reserved information)
	Invalid information (such as branch number not existing, teller information not existing, incorrect exchange rate value, etc.)
	Character length does not meet requirements, is too long or too short (ID number is not 15/18 digits)
	Others

Secondary Classification of business-type errors (6)	Secondary classification code	**DO (Duplicate Operation)**
	Repeat transactions	The same transaction request has been accepted or is currently being accepted; duplicate submissions are prohibited
		The relevant business status already meets the requirements, so there is no need to reconfigure it repeatedly
		Relevant business information or data already exists, so there is no need to enter it again
		Repeated operations are not allowed in business operations, and duplicate applications are prohibited
		Others

Secondary Classification of Business-Type Errors (7)	Secondary classification code	**TM (Time)**
	Time and deadlines	Not during service hours (such as market closure, end of day)
		Suspend business requests (such as during business closures or system maintenance) during designated periods.
		The verification code has expired and become invalid (such as web verification codes, mobile verification codes, SMS verification codes, etc.)
		Expired (e.g., transaction invitation expired, bank card expired)
		User operation interval timeout (requires reoperation or automatic exit)
		Time elements do not meet requirements (e.g., the time period must not exceed 90 days, and the start time must not exceed the end time)
		Others

Secondary Classification of Business Errors (8)	Secondary classification code	**RC (Risk Control)**
	Risk control	For risk monitoring, the trade was temporarily suspended
		Account status prohibited from trading (reported lost, frozen, or other statuses)
		Blacklist, no trading
		Others

Secondary Classification of Business Errors (9)	Secondary classification code	**BL (Business Logic)**
	Business logic	No corresponding business logic (e.g., transfers between accounts are not allowed, and the other account cannot be transferred in, etc.)
		Skipping steps according to business processes, not following normal procedures
		By business process, the previous execution step has not yet been successful or completed
		The relevant business is not activated/not opened/not supported (for example, the two funds applied for cannot be converted)
		Illogical or invalid requests, refusal of transactions (such as closed transactions that cannot be reversed)
		When a business anomaly is detected, it is necessary to forcibly jump to a specific process for execution
		Business-related status does not meet requirements, cannot trade (account has been closed, cannot trade)
		If you fail to apply for virtual teller or other business resources, the transaction cannot continue
		Others

Examples of secondary classifications of technical errors

Technical Errors Secondary Classification (1)	Secondary classification code	**NA(N/A)**
	Uncertain trading results	Response timeout (if the timeout still determines the transaction result, it is not included)
		Cannot identify transaction results from response messages
		Transaction processing interrupted abnormally with uncertain outcomes (such as manual abortion or abnormal interruption of processes/threads)
		Others

ndar	Secondary classification	**HW (Hardware)**

	code	
	Hardware errors	Cannot identify hardware
		Hardware read/write errors (such as IC card read/write errors)
		No correct hardware driver was detected
		Hardware models and parameters do not meet standards
		Others

Secondary Classification of Technical Errors (3)	Secondary classification code	**DC (Data Context)**
	Data content is relevant	Formatting errors (including insufficient data length, unsupported characters, etc.)
		Missing data items and technical parameters
		Incomplete data items, corrupted unstructured data, and incomplete technical parameters
		Data items are invalid, technical parameters are invalid
		Data items are redundant
		Verification fails (e.g., MAC fails check, signature is invalid, etc.)
		Others

Technical Errors Secondary Classification (4)	Secondary classification code	**IO(I/O)**
	Reading and writing	Read/write file errors
		The document is corrupted
		Corresponding files cannot be found (parameter files do not exist, so configuration information cannot be obtained)
		File format error
		Errors during file processing (such as copy errors, deletion errors, etc.)
		Others

Secondary [illegible]	Secondary classification code	**DB (Database)**
	Database-related	Database connection failed
		Database operation anomalies (abnormal returns for addition, deletion, modification, or query)
		Database transaction execution is abnormal

		Others

Technical Errors Secondary Classification (6)	Secondary classification code	**NC (Network Communication)**
	Network communication	Unable to establish a connection
		Sending failed
		Reception failure (if no data is returned)
		Response timeout (if timed out and the transaction result cannot be determined, it is not included)
		Long-term connection maintenance (e.g., long-term network resource usage causing abnormalities)
		Wireless network connection failure (such as call failure, etc.)
		Others

Technical Errors Secondary Classification (7)	Secondary classification code	**SS (Security Service)**
	Security services	Secure nodes are related
		Unable to find the corresponding security service (e.g., security service not started or incorrect startup method, etc.)
		Encryption errors
		Decryption errors (such as incorrect decryption result format)
		Others

Technical Errors Secondary Classification (8)	Secondary classification code	**CM (Components or Modules)**
	Component modules	Cannot call components/modules
		Error in the controls
		Adapter-related errors
		Detects that the application or component service to be called is closed or not in service state
		Other similar ones

ification	Secondary classification	**FC (Flow Control)**

	code	
	Flow control	Exceeding traffic limits
		No available idle processes/threads, transaction rejection
		Exceeding the maximum number of connections
		Others

Secondary classification of technical errors (10)	Secondary classification code	**DO (Duplicate Operation)**
	Repeat transactions	The same service, process, or thread has already started or is running; repeated triggers are prohibited
		Technically, the relevant tasks have been completed, and repeated operations are prohibited
		Technically, the relevant state already exists, and regardless of the status, repeated operations are prohibited
		Technically, repeated operations are not allowed, and duplicate submissions are prohibited
		Others

Technical Error Secondary Classification (11)	Secondary classification code	**TL (Technical Logic)**
	Technical logic	There is no relevant technical logic or corresponding components, or application features are only available to specific platforms
		Technically, there are step-by-step handling and skipped operations in abnormal processes
		Technically, there is step-by-step processing, where the previous execution process is not yet successful or incomplete
		Corresponding technical services are not activated/not opened/not supported
		It does not comply with technical logic, or the request is invalid and support is refused
		If a technical anomaly is detected, a forced jump to a specific step is required
		Technically, the relevant status does not meet requirements and other operations cannot be performed (such as repeated serial numbers, etc.)
		Others

Technical Errors Secondary	Secondary classification code	**AT (Absolute Technique)**
	Purely technical error	Initialization failure (such as failure to initialize variables, objects, or messages, etc.)
		Java's built-in class exceptions (such as dividing 0, outbound array subscripts, etc.)
		Resource release is abnormal
		Failed to apply for shared memory
		Failed to generate serial number
		Others

2.10.6 Principles for writing misinformation

Each error code contains corresponding error information, providing a brief and accurate explanation of the error content.

Error information writing should follow these principles:

1. **Error information specifies the ultimate object.** Error information should accurately specify the decidable ultimate object. For example, when interface mismatch is detected, if the error is due to missing required fields, the error message should not simply state "Required field is empty"; instead, specify which required fields are empty

2. **Error information should specify the specific cause.** Error information should be as specific as possible and not too vague. For example, "Account control status does not allow deposit", it should specify what state caused the deposit to be prohibited; For example, "Other errors, transaction failure"—the error should be specified

3. **Error information should be adapted to end users.** Error information should be adapted to end users and support multilingual and multi-legal entity scenarios. For example, it cannot display "Please call

95533 for consultation" on the Sino-German Bank channel, nor display Chinese error messages overseas

2.10.7 Error information escape mechanism

Error codes and error information are defined and written by the backend component project team. The front-end channel processes information clientization based on backend error information, which is called error code escaping. The escaped backend error information is considered front-end error information.

For error codes and messages generated by backend components, the backend should distinguish escaping prompt identifiers according to different situations. Prompt identifiers are divided into static and dynamic. The frontend handles different types based on the prompt identifier:

Static mode: After receiving the backend response message, the channel side searches the channel configuration database based on the error code, and when it detects the prompt indicating the error code as "static," the frontend error information is displayed directly on the interface

Dynamic mode: After receiving the backend response message, the channel side searches the channel configuration database based on the error code. If it detects the error code indicating "dynamic," the front-end error information in the database is combined with the dynamic parameter values in the response message and displayed on the interface

2.11 Serial number (global event tracking number) code

The serial number, here specifically refers to the global event tracking number, generated from event trigger points **(event trigger or rule trigger)**, unique **globally, shared throughout the entire process,**

and continues until the event is completely completed. In other words, the same event inherits the same serial number when processing across platforms, components, or transactions; If a single event triggers multiple child events, the child events also inherit the same serial number

2.11.1 The purpose of serial numbers (global event tracking numbers) codes

Serial number (global event tracking number) is used to uniquely identify and track an event (transaction) and the sub-events (sub transactions) triggered by it

2.11.2 Serial numbers (global event tracking numbers) are generated independently

The generation of serial numbers (global event tracking numbers) is done independently by each machine. Each machine is assigned a globally unique machine number, following serial number coding rules to ensure global uniqueness.

2.11.3 Serial number (global event tracking number) coding rules

Serial number (global event tracking number), using **a** 4-segment **25-bit** encoding method, as shown in the figure below:

The serial number (global event tracking number) uses a four-segment, 25-bit code			
6 people	**3**	**10th place**	**6 people**
Secure node number (Security Entity)	**Within the same security node Machine**	**Represented by an integer Year, month, day, hour, minute, second**	**Serial number**

	number		
Defined by security architecture	**Arranged by the responsible party who obtained the security node number**	**This integer is unrelated to the development platform**	**Generated by the machine itself**
Numbers first, letter compatibility	**Numbers first, allowing letters**	**Pure numbers**	**Pure numbers**

First paragraph: 6 bits long, which is the security node number, usually indicating the corresponding number of the security entity that generated the serial number. Here, the security node number directly references the secure node code

Paragraph 2: 3 characters long, representing the machine number within the same security node. This machine number is orchestrated by the responsible party who obtained the security node number and ensures its uniqueness (unique within the same security node, prioritizing numeric orchestration, and **only using uppercase letters after 999 is used**). Basic requirements: each server must have at least one unique machine number; multiple machines are prohibited from sharing the same machine number. If a machine deploys multiple applications (components) simultaneously, multiple machine numbers can be assigned (in this case, be careful not to use them cross-control).

The first segment is the secure node number, and the second segment is the machine number, supporting local reading. The file directory (path) can be accessed via local environment variables.

Third paragraph: 10 bits long**, representing the number of seconds, which is an integer representing the** number of seconds from the CUT **(**Coordinated Universal Time**) time** on **January 1**, 1970**, 00:00:00** (also known as **UNIX Epoch/POSIX time**) to the current moment. To obtain this integer, you can first calculate **the total number of days from midnight on New Year's Day 1970** to the current date, then perform the relevant conversion and add the total number of seconds that have passed

on that day. If you use the built-in functions built into the development platform to directly or indirectly obtain this integer, you need to thoroughly test the maximum years supported by the built-in functions. According to the calculation, the **maximum value of this 10-digit integer "**9999999999**" corresponds to the moment** of November 21**, 2286**, **01:46:39 (Beijing time GMT+8).**

Fourth paragraph: 6 bits **long**, serving as the serial number, generated by the machine's own serial number generator, **starting from 0** and increasing by 1 **after each serial number retrieved**, repeating **from 0** until **the 999999 is reached**. The serial number should be independent of time; it is forbidden to reset the serial number at a certain point because the machine's system time may be manually adjusted or automatically synchronized, resulting in the same time

Note: Another encoding method uses the third segment of encoding to milliseconds (i.e., 13 bits), and the fourth segment uses only 3 digits. The drawback of this method is that the probability of repeated serial numbers is much higher. The reasons are as follows: the machine's system time is manually adjusted or automatically synchronized, and once the system time jumps back, duplicate times occur; If the subsequent serial number has only 3 digits, the cycle period for these 3 digits will be very short (for busy servers, this cycle may be less than 1 minute), making repeated serial numbers very likely. If the first encoding method is used, the loop period for the last 6 digits is relatively long (usually longer than a day), and the system time is adjusted and the backward jump is generally less than 1 hour, making the chance of repeated serial numbers very low

Generally, since uppercase "**O**" and the number "**0**" can cause visual confusion, it is recommended to avoid or minimize the use of uppercase "**O**" as an encoding element to avoid **simultaneous occurrence of both**

"O" and **"0" within the same encoding** system

From the above, it can be seen that if the computer system time is incorrect, duplicate serial numbers may occur. To be cautious, the component generating this "serial number" must have a "self-check system time" function: if abnormal time changes causing possible repeat times, relevant prevention mechanisms are activated to prevent duplicate serial numbers from being generated; and related alerts are issued.

2.11.4 Serial number requirements in serial numbers (global event tracking numbers) coding

The fourth segment of the serial number (global event tracking number) must be **incrementally increasing in one direction and reset to zero after reaching the maximum value. In any** case—service restart, system restart, or restart after power outage—the serial number must be incrementally advanced in one direction. For example, the following situations are prohibited: For example, if the current serial number is 120000, once restarted, the serial number starts from 0 again

It is recommended to use the "record next footprint" method to ensure the unidirectionality of the serial number. That is: if the current serial number is 120000 and the step length is 10000, then the "next footprint" is 130000. This value of 1300000 is fixed and recorded on the disk. Normally, the serial number increases continuously from 120000 incrementing forward. If the serial number is interrupted due to reboot or other reasons, after reboot, the "next footprint" value is read from disk, with the serial number starting directly from 130000 and skipping the remaining values between 120000 and 130000. After each step is completed, the serial number corresponding to the "next footprint" is

updated to the disk. The length of the stride can be adjusted by the application according to actual needs; it should not be too large or too small.

2.12 Types and codes of transactions

2.12.1 The purpose of the type of transaction

Types of transactions are classified based on business or technical needs.

For example, from a technical perspective, transactions are classified for prioritization and flow control, such as prioritizing certain types of transactions during peak or busy periods.

2.12.2 Classification of transaction types

Transaction types can generally be categorized by priority, whether accounting is involved, transaction response time, and transaction direction.

2.12.2.1 Classified by priority

During transaction processing, various priorities or other special privileges are provided according to business or technical needs. Transaction priority is divided **from low to high into ten levels, from 0 to 9**

Transaction priority	Explanation
0	Eight levels in regular trading From low to high
1	

2	
3	
4	
5	
6	
7	
8	Traffic Exemption Transactions (see below)
9	Senior Priority Deal (see below)

① **Regular transactions** do not fall under the categories of traffic control-free trading or special priority trading

② **Traffic-free transactions** are not subject to traffic control and follow a green channel on the trading line. For example, account loss reports should be considered this type of transaction

③ **Special-grade priority transactions** have higher priority and are not subject to traffic control. If a queue is encountered on the transaction line, it can skip the queue and process it directly. For example, if a transaction is initiated to the backend to perform a special task (such as urgently aborting a batch transaction in progress), special priority permission can be granted to that transaction; Similarly, some urgent positive transactions can also have special priority permission

2.12.2.2 Classified by whether accounting is involved

① **Book-based transactions involving changes in the** quantity of funds or bonds or other valuable entities

② **Accounting collaboration transactions** refer to preceding or subsequent transactions that must be completed when completing an accounting transaction. If these transactions are missing, the corresponding transaction cannot be completed normally. For example, transfer fee inquiry transactions conducted before transfers. Generally,

if only accounting transactions are given high priority but not corresponding high priority for accounting collaboration transactions, the set high priority for accounting transactions is invalid

③ **Non-accounting transactions**, such as inquiries, contracts, registrations, registrations, authorizations, authentication, and management

Generally, **accounting collaboration transactions and account-independent transactions** are both called non-accounting transactions

Note: If a transaction is in different business scenarios, whether it is an accounting collaboration transaction or an account-independent transaction, the corresponding transaction type code should be switched according to the business scenario (and the transaction code code can be used in the same way).

2.12.2.3 Classified by transaction response time

① **Real-time trading** and real-time response trades generally have corresponding timeout settings

② **Near real-time trading** has no clear boundary between real-time and non-real-time trading. This type of trading generally has the following characteristics: **predictable response time** (the error between actual response time and estimated time is calculated in minutes), T**+0** (not the next day).

③ **Non-real-time trading**, such as common end-of-day batch trading

2.12.2.4 Classified by trading direction

① **Positive trading**, generally a positive trade

② **Margin trading can be** further divided into automatic and manual marginal trading

2.12.3 Transaction type coding rules

The transaction type code uses **a** 6-digit flag code method, as shown in the figure below:

(For the reserved two flag bits, fill in the default value **0**.))

Transaction types use six-digit codes					
1	2	3	4	5	6
First place	Second place	Third	Fourth place	Fifth place	Sixth place
Special classification Landmark position	Whether accounting marker positions are involved	Response time marker position	Trading direction Landmark position	Reserve Landmark position	Reserve Landmark position

Explanation of transaction type codes:

Landmark position	Classification	Value taken	Explanation
First place	Special classification	**0~7**	Eight **levels** in regular trading
		8	Traffic-free transactions
		9	Premium priority trade
Second place	Classification by whether accounting is	**0**	Accounting transactions
		1	Accounting collaboration transactions
		2	Transactional non-accounting
Third	Classified by response time	**0**	Real-time trading
		1	Near real-time trading
		2	Non-real-time trading
Fourth place	Classified by transaction direction	**0**	Positive trading
		1	Automatic margin trading is available
		2	Manual Stock Trading

		3	Cancel the transaction
		4	Re-enter the transaction with the voucher
		5	Positive trade response message query trade
		6	Correct trading response message query trade
Fifth place		**0**	Non-error-adjusted transactions
		1	Correction of errors in red-letter transactions
		2	Authorize predictive trading
Sixth place		**0**	Reserve and fill in **0**

3. Technical code-naming standards

The purpose of this naming standard is to provide a technical code naming method that is relatively vivid, easy to read, simple, clear, and consistent in principle, thereby improving the readability of technical code and system maintainability.

The global technical code naming referred to here refers to naming general "identifiers," generally including the contents listed in the table below:

Identifiers	
Programming identifiers	Variables, macros, functions, procedures, methods, classes/structures, modules, etc.
Database identifier	Database names, tables, columns/fields, primary/foreign keys, indexes, views, triggers, etc.
Message identifier	Header element

3.1 Six basic rules for naming identifiers

The six basic rules for naming identifiers are: word choice rules, length rules, spacing rules, case and case rules (case rules), numeric rules (numbering), and consistency rules.

3.1.1 [Naming Rule 1] Word Usage Rules

All identifiers must use English words or abbreviations; using Hanyu Pinyin is not recommended.

The wording of identifiers should be concise and clear, avoiding ambiguity.

When using English word abbreviations, prioritize the abbreviations specified in enterprise-level data dictionaries or metadata. If special abbreviations are required in identifier naming, annotations or explanations must be made at the beginning of the source file or where the identifier appears.

3.1.2 [Naming Rule 2] Length Rule

Identifiers must comply with the principle of "maximizing information and minimizing length."

It is generally recommended to use long names, as long names better convey meaning, but they should be "as short as possible," generally no more **than 30** characters. Single-byte naming is only allowed for local loop variables within functions, such as i, j, k, m, n, x, y, z, etc

3.1.3 [Naming Rule 3] Spacing rules

If an identifier contains multiple words, it must be separated by either a capital first letter or an underline. In the same code fragment or the same context, only one method is generally required.

COBOL uses "connectors" instead of underscores

举例，大写首字母间隔法如 ObjectProxy/ ServiceManager/SrvMgr 等；下划线间隔法如 object_proxy 等。

3.1.4 [Naming Rule 4] Case Rules

Identifiers are prohibited from being distinguished by case alone, meaning two or more identifiers with different case cases are prohibited from appearing within the same scope or visible area.

For example: int x and int x; void foo() and void FOO(), etc

3.1.5 [Naming Rule 5] Number Rules

Numbers in identifiers can only serve as auxiliary labels. When using numbers in identifiers, it must be ensured that the letters fully convey meaning, and that numbers are only used for numbering or specific meanings.

It is especially emphasized that if the number of letters in an identifier is less than **5** characters, the use of numbers is prohibited!

For example, similar variable or function names are prohibited: a001, void b002().

3.1.6 [Naming Rule 6] Conformance Rule

The names used for various identifiers (data names, variables, constants, database table names/field names, etc.) should follow the corresponding names published in the enterprise-level data dictionary. All forms of identifiers expressing the same meaning must strictly use the same name, but prefixes or suffixes may be added to distinguish between special situations.

In programs or systems, when referencing data items in the data dictionary, it is recommended to add prefixes ("8-bit module name **+** 1-bit separator", **COBOL** separator as the connector '**-**', **C/Java** separator as the underline '**_**').

3.2 Naming of programming identifiers

Under the premise of strictly adhering to the "Six Basic Rules for Identifier Naming," the naming of programmed identifiers follows the detailed rules

3.2.1 Module

A module generally refers to a collection of program statements (i.e., a collection of program code and data structures) that can perform

specific functions or services in programming design and development; or refers to independent program units that can be handled by compilers, assemblers, and similar programs. They generally have three basic characteristics: independence, high reusability, and collaboration.

Module names are generally **composed of 1** to **8** uppercase letters. For modules with publicly exposed interfaces and frequent reuse, it is recommended to design a name that is easy to remember and recognize. For function naming and class naming below, it is recommended to use the module name as a prefix to facilitate identification of specific module objects

3.2.2 Classes/interfaces

(Here, 'interface' specifically refers to **the** interface **in Java**.)

It is generally a noun or "adjective + noun" structure, using a mixed case system, capitalizing the first letter of each word.

Generally, underscores should not appear in the names of classes/interfaces.

For **classes in C++**, you can use the capital **letter C** as the initial

If a module name has already been defined, it is recommended to use the module name as a prefix.

3.2.3 Function/Method

It is usually a verb or "verb + noun" structure, fully expressing the function or service provided by the function/method.

The first letter of the first word is lowercase, and the first letter of each following word is usually capitalized.

If a module name has already been defined, it can be used as a prefix where applicable, without the restriction of the "first letter of the

first word" in lowercase.

In **C**, if the function interface is published as a dynamic library, it's best to use the module name or other shorthand tags as a prefix

3.2.4 Variables/parameters

It is generally a noun or an "adjective noun" structure.

No mandatory requirements are required; simply following the "**Six Basic Rules for Identifier Naming**" mentioned in the previous section is sufficient

3.2.5 Constant

All letters of the words are capitalized, separated by underscores.

Naming example:

```
const int MAX_ARRAY = 8;
```

3.2.6 C/C++ proprietary identifier

When programming in **C/C++**, the naming of relevant identifiers must follow the general rules above; other rules are as follows

3.2.7 Macro

Macro: All letters of each word are capitalized, separated by underscores

Naming example:

```
#define MAX_SLOT_NUM 0x08
```

3.2.7.1 Structure

Structure: All letters of each word are capitalized, and words are connected by underlines

Naming example:

```
typedef struct LOCAL_SPC_TABLE_STRU
{
    char val;
    int SpcCode[MAX_SLOT_NUM];
}LOCAL_SPC_TABLE;
```

3.2.7.2 Enumeration and enumeration members

Enumerate the letters of each word, with all letters capitalized, separated by underscores

Enumerate members, capitalize all letters of words, and separate words with underlines. The first word of each member must be the same Naming example:

```
typedef enum
{
    LAPD_MDL_ASSIGN_REQ,
    LAPD_MDL_ASSIGN_IND,
    LAPD_DL_DATA_REQ,
    LAPD_DL_DATA_IND,
    LAPD_DL_UNIT_DATA_REQ,
    LAPD_DL_UNIT_DATA_IND
}LAPD_PRMV_TYPE;
```

3.2.8 JAVA proprietary identifier

When programming in **Java**, besides following the above general rules, the naming of relevant identifiers is as follows

3.2.8.1 Package

It consists of words or word abbreviations, all lowercase, and is generally a top-level domain. For this system, it is recommended to use "**com.ccb**" as the prefix for **the Java Package**

Words used in naming generally do not use plural numbers.

例如:使用:**com.ccb.servlet**;不使用:**com.ccb.servlets**。

Note: The current standard domain name for CCB Internet is **www.ccb.com**, so **the package** structure uses **com.ccb** as the prefix

3.2.8.2 static 与 final 变量

All letters of the words are capitalized, separated by underscores.

3.3 Naming of message identifiers

Under the premise of complying with the "Six Basic Rules for Identifier Naming," the naming of message identifiers should be done according to the following detailed rules

To minimize the impact on transaction execution efficiency and network throughput, the principle of "shortening as much as possible" should be followed when naming message elements, as long as readability is not compromised. At the same time, the length of message element names should be strictly controlled, aiming not to exceed half of the total transaction packet length

3.3.1 Packet header identifier

General fields in packet headers (mostly technical elements) are named with "**SYS_**" as the prefix

Other extended identifiers for messages are defined and named in

detail according to application architectures and other criteria.

3.3.2 Message text identifier

Identifiers for message text must not use "**SYS_**" as a prefix

The same transaction element must be named consistently across different business transactions. For example, "transfer amount" must be named the same in transactions such as "intrabank transfer" and "cross-bank transfer."

4. Coding standards for multilingual and multi-entity codes

4.1 Coding standards for multiple language codes

Generally, all languages use the "**two-digit language code**" encoding method; If the same language has differences across regions, the "**two-digit language code - two-digit area code**" (5 digits in total, including the middle bar "-" symbol) is used

The language codes in the table below are encoded based on internationally commonly used multilingual code writing conventions. The multilingual codes listed at the top are those of countries or regions with CCB business, while those listed at the bottom are multilingual codes from other common countries or regions.

Language Code (Identifier)	English name	Chinese name
zh-cn	Chinese (Simplified)	Chinese (Simplified)
zh-hk	Chinese (Hong Kong)	Chinese (Hong Kong Special Administrative Region, China)

zh-tw	Chinese (Taiwan)	Chinese (Taiwan, China)
zh-ug	Uigur	Uyghur language (Chinese Uyghur ethnicity)
In	English	English (Standard)
EN-US	English (United State)	English (USA)
en-au	English (Australia)	English (Australia)
en-ca	English (Canada)	English (Canada)
en-za	English(South Africa)	English (South Africa)
of	German	German (standard)
de-lu	German (Luxembourg)	German (Luxembourg)
fr-lu	French (Luxembourg)	French (Luxembourg)
and	Japanese	Japanese
is	Korea	Korean (Korea)
vi	Vietnam	Vietnamese
ru	Russian	Russian (Russia)
on-EU	Arabic (United Arab Emirates)	Arabic (United Arab Emirates)
on-sa	Arabic (Saudi Arabia)	Arabic (Saudi Arabia)
ar-kw	Arabic (Kuwait)	Arabic (Kuwait)
ar-qa	Arabic (Qatar)	Arabic (Qatar)
AR-IQ	Arabic (Iraq)	Arabic (Iraq)
AR-IN	Arabic (Iran)	Arabic (Iran)
AR-EG	Arabic (Egypt)	Arabic (Egypt)

cs	Czechic	Czech
da	Denmark	Danish
Contact	Dutch (Holand)	Dutch (standard)
fi	Finnish (Finland)	Finnish
it	French	French (standard)
fr-be	French (Belgium)	French (Belgium)
fr-ch	French (Switzerland)	French (Swiss).
The	Hellenic (Greek)	Greek
hu	Hungarian	Hungarian
it	Italian	Italian (standard)
pl	Polish	Polish
en	Spanish	Spanish (standard)
Es-ar	Spanish (Argentina)	Spanish (Argentina)
es-cl	Spanish (Chile)	Spanish (Chile)
sv	Swedish	Swedish
th	Thailand	Thai
tr	Turkish	Turkish
.....	.....	.....

Explanation:

1: Traditional Chinese in Taiwan (Traditional Chinese) differs significantly from Traditional Chinese in Hong Kong (for example: differences in individual character writing forms, vocabulary usage, spoken habits, etc.). According to international software translation

conventions, these two types of Traditional Chinese are treated separately

2: In **May** 1976, Singapore officially issued a revised version of the "Simplified Character Table," which is now fully consistent with Chinese mainland's "Simplified Character Table." Therefore, Singaporean Chinese is not treated as an independent language

3: Currently, some CCB ATMs in Xinjiang use the Uyghur script. For now, only the language codes of Uyghur minority scripts are listed

4.2 Coding standards for multi-entity codes

Note that the meanings of "entity" and "legal entity" in this chapter differ.

The definition of "entity" is as follows:

Definition 1: Multi-entity applications allow customers to share the same hardware resources, provide shared applications and database instances, and allow customers to configure applications according to their own needs, giving customers the feeling of running in a dedicated environment.

Definition 2: A tenancy is a specific organization that leases a multi-entity SaaS solution. Typically, an entity includes a group of users who are stakeholders of the organization.

Definition 3: Tenancy ID is used to identify specific organizations and distinguish business processes, user interfaces, and data. The entity ID is reflected in system data flow and data storage.

The "entity code (entity ID)" mentioned in this section is mainly used to distinguish legal entities, countries, and regions. Corresponds to the domain in the data dictionary—**multi-entity identifier**

"Entity Code (Entity **ID**)" uses the following encoding rules:

"Entity Code (Entity **ID**)" **uses a 2-segment, 5-digit code**	
2 digits (uppercase letters).	**3rd digit (number)**
Country or region code	Legal entity code
Domestic values are CN; for overseas and subsidiaries, refer to the specific published values	Refer to the specific published values